Program and Branch Name:
B.Tech CSE – A

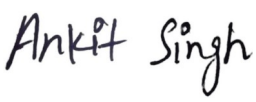
PBL Final Report

Field	Details
Course Name	Data Structures and Algorithms - II
Course Code	CCSE0301
Faculty Name	Mr. Shamshad Ali
Assignment Type	Individual Assignment
Student(s) Name	Ankit Singh
ERP ID (I Year) / Roll No. (II Year onwards)	2501330100075
Team/Group No. (Group/Capstone only)	N/A – Individual Assignment
Problem Statement / Project Title	Smart Student Progress Monitoring Dashboard
SDG(s)	SDG 4 – Quality Education
Reporting Month	Month 3 – Final
Date of Submission	29-10-2026

Student Declaration

I certify that this assignment is my own work, with all sources appropriately acknowledged. I understand that a 0% mark and further academic penalties may apply if the content is found to be plagiarised or inappropriately AI-generated.

Student Name & Signature: Ankit Singh



Date: 29-10-2026

1. Student Details

S.No.	Student Name	ERP ID / Roll No.	Role / Contribution
1	Ankit Singh	2501330100075	Sole contributor: design, coding, testing and report writing

2. Executive Summary

This project builds a Smart Student Progress Monitoring Dashboard that turns a class's marks, attendance and assignment scores into a progress score, ranks students by risk, links each at-risk student to a similar and stronger peer, plans study hours, and schedules mentoring. It applies trees and heaps, graph algorithms, dynamic programming, and backtracking with branch and bound to five tasks, and studies advanced trees and heaps as a design option. A Python prototype was run on a 12-student sample and on synthetic sets of 1,500 and 20,000 records, and every result matched an independent check.

3. Introduction

Teachers usually see a struggling student only after an exam has been lost. Marks, attendance and assignments are recorded separately, so the warning signs are spread across registers and spreadsheets. A dashboard that combines them can show early who needs help, which supports SDG 4 (Quality Education) by keeping students from falling behind unnoticed.

The objective is to apply each topic of the course (trees and heaps, graphs, dynamic programming, backtracking and branch and bound, and advanced trees) to one practical problem and keep every algorithm explainable. The scope is a console prototype on synthetic data. It excludes real student records, machine-learning prediction and any link to a college ERP.

4. Problem Definition

Problem statement: given each student's marks, attendance and assignment results, compute a progress score, rank and flag at-risk students, find the closest suitable peer or mentor, plan study hours, and schedule mentoring without clashes.

- **Objectives:** build the prototype, test each algorithm against an independent check, and document the results.
- **Target users:** faculty mentors, class coordinators and students.
- **Constraints:** scores are normalised to 0–1; similarity distances are never negative; study time and mentoring slots are limited; data is synthetic.
- **Success criteria:** the most at-risk student is always ranked first, Dijkstra agrees with Floyd–Warshall, the study plan, slot count and mentor round each equal a brute-force check, and memoization removes repeated work.
- **Linked SDG:** SDG 4 – Quality Education.

5. Industry / Application Overview

Colleges and online platforms use learning analytics and early-warning systems to find students at risk. Albreiki, Habuza and Zaki (2023) turn student records into a graph through a distance-based adjacency matrix and feed its topological features to machine learning. Trask et al. (2024) put students, videos and assessments into one heterogeneous graph for online courses.

The gaps are clear. The first approach links students only, needs $O(n^2)$ matrix space, depends on a distance threshold and relies on training that is hard to explain. The second suits online-course data and does not say what a weak student should do next. This project uses an adjacency list, a documented threshold and plain classical algorithms, and it gives actionable outputs: a priority list, a nearest mentor, a study-hour plan and a clash-free mentoring schedule.

6. Module 1: Trees and Heaps

Concepts applied: binary max-heap, heap sort, AVL tree and in-order traversal. A max-heap ordered by risk (100 minus the progress score, ties broken by lower attendance) keeps the most at-risk student at the top, and insertion and removal cost O(log n). Heap sort ranks the whole class in O(n log n). An AVL tree keyed by progress score rotates after each insertion to stay balanced, so sorted display and range queries cost O(log n + k).

Output: heap sort ranked S10, S07, S05, S06 and S11 as the five highest-risk students, and the full ranking equalled Python's built-in sort. The AVL tree on 12 students had height 4 after 7 rotations, its in-order traversal was sorted, and the range query for scores 40–60 returned S07 (43.9), S05 (50.5) and S06 (55.4), the same as a plain filter. On 20,000 synthetic records (seed 7) the heap was built in about 0.03 s, one removal took about 0.01 ms, heap sort took about 0.1 s and was correct, and the AVL height was 17 against an ideal of about 15. The AVL range query (about 8 ms) was close to a list filter (about 12 ms), so its advantage appears only with larger data or repeated queries.

7. Module 2: Graphs and Graph Algorithms

Concepts applied: adjacency list, BFS, DFS, connected components, Dijkstra and Kruskal. Each student is a node, and two students share an edge when the Euclidean distance between their normalised (marks, attendance, assignment) vectors is at most 0.15; that distance is the edge weight and is never negative. Subject nodes link a student to weak subjects. The list needs O(V + E) space, traversals and component search run in O(V + E), Dijkstra in O((V + E) log V) and Kruskal in O(E log E). The progress score is 0.5 × marks + 0.3 × attendance + 0.2 × assignments, and students below 55 are at risk.

Roll	Name	Marks	Attend.	Assign.	Score	Status
S01	Aarav	88	94	90	90.2	OK
S02	Bhavna	82	90	85	85.0	OK
S03	Chirag	76	85	80	79.5	OK
S04	Divya	91	96	93	92.9	OK
S05	Esha	45	60	50	50.5	At-risk
S06	Farhan	52	66	48	55.4	OK
S07	Gauri	38	55	42	43.9	At-risk
S08	Harsh	70	80	72	73.4	OK
S09	Isha	64	75	66	67.7	OK
S10	Jatin	30	40	35	34.0	At-risk
S11	Kavya	58	70	60	62.0	OK
S12	Lakshya	85	88	87	86.3	OK

Output: the 12-student sample gave 14 edges and two components, eleven connected students and S10 alone, so S10 is both at-risk and isolated. BFS and DFS from S05 reached the same eleven students. Dijkstra matched Floyd–Warshall for every node, and Kruskal returned 10 edges of total weight 0.915, which is V minus the number of components (12 − 2). Mentors were the nearest student with a score of 80 or more:

At-risk student	Score	Nearest mentor	Mentor score	Path distance
S05 (Esha)	50.5	S02	85.0	0.636
S07 (Gauri)	43.9	S02	85.0	0.753
S10 (Jatin)	34.0	None	–	no path

S10 has no path to any mentor, which correctly signals that this student needs direct staff attention. The result depends on the threshold: at 0.20 or higher S10 joins the main cluster and nobody is isolated, so faculty should tune it on real data. The 0.15 threshold and the score weights are design choices, not official values. Prim, Bellman–Ford, matrix-chain multiplication and topological sort were studied but not coded, because the dashboard has no negative weights, no chained matrices and no prerequisite data; Floyd–Warshall was used only to cross-check Dijkstra.

8. Module 3: Dynamic Programming

Concepts applied: 0/1 knapsack, LCS and memoization. Each study topic has an hour cost and an assumed marks gain. The knapsack table has O(n × W) cells and gives the exact optimum. LCS, O(m × n), checks for near-duplicate roll numbers, and memoization caches semester averages that the dashboard requests repeatedly.

Topic	Hours	Assumed gain	Chosen
DSA	4	12	Yes
DBMS	3	8	Yes
OS	2	5	No
Maths	3	9	Yes
Project work	5	10	No

Output: with 10 study hours the table chose DSA, DBMS and Maths (10 hours, gain 29), and checking all 32 subsets confirmed 29 is the maximum. LCS of two test strings gave 6, equal to brute force, and LCS of S0123 and S0132 gave 4 of 5 characters, which flags a probable typo. For 12 students and 6 semesters, memoization made 72 computations instead of 360 over five dashboard refreshes.

9. Module 4: Backtracking and Branch and Bound

Concepts applied: backtracking, graph colouring and branch and bound. Two at-risk students who share a weak subject need the same mentor, so they cannot take the same mentoring slot. This gives a conflict graph (S05–S06, S05–S07, S05–S10, S06–S10, S07–S10, S10–S11), and each slot is a colour. Backtracking assigns slots one student at a time and undoes a choice when it causes a clash. It found a valid 3-slot schedule (S10 in slot 1; S05 and S11 in slot 2; S06 and S07 in slot 3) and showed that 2 slots are impossible, because S05, S06 and S10 conflict pairwise. A brute-force check over all assignments agreed.

For travel, a mentor leaves the office, visits Block A, Block B, Block C and the Library, and returns, using fixed synthetic travel times in minutes. Branch and bound drops any partial route that already costs at least the best full route found. It returned a 26-minute round (Office → Block A → Block B → Block C → Library → Office), equal to the brute-force minimum over all 24 orders, after exploring 62 partial routes and pruning 17. Hamiltonian cycles and sum of subsets follow the same pattern and were studied only.

10. Module 5: Advanced Trees and Heaps (Design Study)

Red-black trees, B-trees, B+ trees, binomial heaps and Fibonacci heaps were studied but not coded, because a class of a few thousand students shows no practical gain. A university-wide student database kept on disk would suit a B+ tree, whose linked leaves answer range queries such as “all students scoring 40–60” efficiently. A red-black tree gives guaranteed O(log n) updates for a live in-memory dashboard. A Fibonacci heap lowers Dijkstra's bound to O(E + V log V) on a very large similarity graph, and binomial heaps allow cheap merging of department-wise priority queues.

11. Final Integrated Solution / Testing and Validation

Integration: the program loads and validates records (LCS flags near-duplicate roll numbers), computes progress scores, ranks students with the heap, keeps the AVL tree for range queries, builds the similarity graph, finds clusters, mentors and the spanning tree, then plans study hours, mentoring slots and the mentor's round. Data handling is kept separate from the algorithms, and the prototype runs from one file (python student_dashboard.py).

Test	Expected Result	Result
Heap sort ranking	S10, S07, S05 first; equals built-in sort	Pass
AVL in-order traversal	Sorted by progress score	Pass
AVL range query, 40–60	S07, S05, S06 = plain filter	Pass
Progress score and risk flag	S05, S07, S10 below 55	Pass
BFS vs DFS from S05	Same set of 11 students	Pass
Connected components	2 clusters; S10 isolated	Pass
Dijkstra vs Floyd–Warshall	Equal distances from S05	Pass
Kruskal MST	10 edges = V − components; weight 0.915	Pass
Mentor search	S05 → S02, S07 → S02, S10 none	Pass
0/1 knapsack, 10 hours	Gain 29 = brute force	Pass
LCS and memoization	LCS 6 = brute force; 72 vs 360 runs	Pass
Slot colouring	2 slots impossible; 3 succeed = brute force	Pass
Branch and bound round	26 min = brute-force minimum	Pass

Benchmark on 1,500 synthetic students (threshold 0.08): 4,607 edges, graph build about 0.30 s, component search 0.004 s, Dijkstra 0.001 s and Kruskal 0.007 s. Building the graph compares every pair of students, $O(n^2)$, so it is the bottleneck for very large classes. Times vary by machine, and all data is synthetic, so the results show algorithmic correctness, not educational effectiveness.

12. Expected Benefits / Learning Outcomes

Mentors get an explainable priority list, at-risk students get a named peer and a study plan that fits their time, and mentoring can be scheduled without clashes, supporting SDG 4. Learning outcomes: choosing the right data structure for each task, analysing time complexity, using dynamic programming only where it pays off, and validating code against independent checks.

13. Challenges Faced

- Defining what a node, an edge and its weight mean; solved with a normalised distance and a documented threshold.
- Equal risk scores made heap order differ from the built-in sort, so ties are broken by attendance and tests compare keys.
- A loose threshold joins everyone into one cluster, and branch and bound needed pruning to stay fast.
- Real student data is confidential, so synthetic data and fixed travel times were used.

14. Future Improvements

- Test with anonymised real records and tune the threshold and score weights with faculty.
- Implement B+ trees and Fibonacci heaps, and speed up graph building instead of comparing every pair.
- Add a menu or web interface with a database, and subject prerequisites using topological sort.

15. Conclusion

The Smart Student Progress Monitoring Dashboard shows how trees and heaps, graphs, dynamic programming, and backtracking with branch and bound combine to solve one educational workflow. The prototype passed all thirteen tests, including a 26-minute mentor round and a 3-slot schedule that equal their brute-force checks. The project improved my understanding of choosing and analysing algorithms for a socially useful problem.

16. References

1. B. Albreiki, T. Habuza and N. Zaki, “Extracting topological features to identify at-risk students using machine learning and graph convolutional network models,” Int. J. Educ. Technol. High. Educ., vol. 20, art. 23, 2023. https://doi.org/10.1186/s41239-023-00389-3

2. T. Trask, N. Lytle, M. Boyle, D. Joyner and A. Mubarak, “A comparative analysis of student performance predictions in online courses using heterogeneous knowledge graphs,” arXiv:2407.12153, 2024. https://arxiv.org/abs/2407.12153

3. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

4. E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959.

5. J. B. Kruskal, “On the shortest spanning subtree of a graph and the traveling salesman problem,” Proc. Amer. Math. Soc., vol. 7, no. 1, pp. 48–50, 1956.

6. R. W. Floyd, “Algorithm 97: Shortest path,” Commun. ACM, vol. 5, no. 6, p. 345, 1962. https://doi.org/10.1145/367766.368168

7. United Nations, “Goal 4: Ensure inclusive and equitable quality education,” SDGs. https://sdgs.un.org/goals/goal4

17. Appendix (Source Code)

Source file and Repository link: https://github.com/ankitsingh87877-rgb/Smart-Student-progress-monitoring-dsahboard

Faculty Evaluation:

To be completed by Faculty in Moodle after each review report submission, by conducting a proper viva-voce or presentation as per the department-level decision.